NAME: C. PRIYANKA

REG NO: 192511074

11) MAP COLORING TO IMPLEMENT CSP

Aim

To develop a Python program to solve the Map Coloring Problem using the Constraint Satisfaction Problem (CSP) approach.

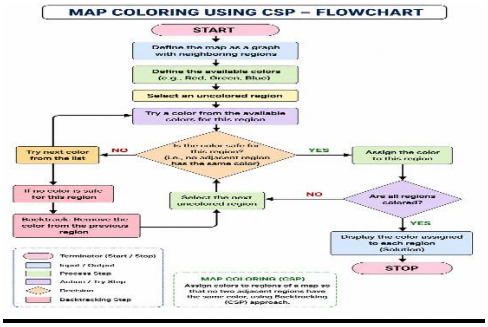
Objectives

- To understand the Map Coloring Problem in Artificial Intelligence.
- To apply Constraint Satisfaction Problem (CSP) techniques.
- To assign different colors to adjacent regions.
- To satisfy all coloring constraints using backtracking.

Algorithm

1. Start the program.
2. Define the map as a graph with neighboring regions.
3. Define the available colors.
4. Select an uncolored region.
5. Assign a color that does not conflict with neighboring regions.
6. If no valid color is available, backtrack and try another color.
7. Repeat until all regions are colored.
8. Display the color assigned to each region.
9. Stop the program.

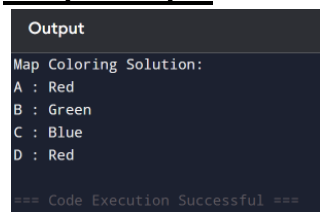
Flow Chart



Python Program

```
graph = {
   'A': ['B', 'C'],
   'B': ['A', 'C', 'D'],
   'C': ['A', 'B', 'D'],
   'D': ['B', 'C']
}

colors = ['Red', 'Green', 'Blue']
result = {}

def safe(node, color):
   for neighbor in graph[node]:
      if neighbor in result and result[neighbor] == color:
         return False
   return True

def solve(node_list):
   if not node_list:
      return True

   node = node_list[0]

   for color in colors:
      if safe(node, color):
         result[node] = color

         if solve(node_list[1:]):
            return True

         del result[node]

   return False

solve(list(graph.keys()))

print("Map Coloring Solution:")
for node in result:
   print(node, ":", result[node])
```

Sample Output



Result

Thus, the Python program to solve the Map Coloring Problem using the Constraint Satisfaction Problem (CSP) was implemented successfully, and valid colors were assigned to all regions without violating any constraints.

Conclusion

The **Map Coloring Problem** is a classic **Constraint Satisfaction Problem (CSP)** in Artificial Intelligence. By using the **Backtracking** technique, the program efficiently assigns colors to all regions while ensuring that no two adjacent regions have the same color.

12) TIC TAC TOE GAME

Aim

To develop a Python program to implement the Tic-Tac-Toe Game using Artificial Intelligence concepts.

Objectives

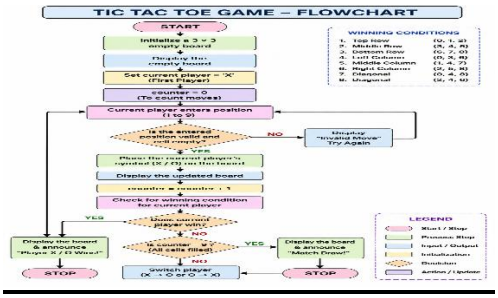
- To understand the working of the Tic-Tac-Toe game.
- To implement a two-player Tic-Tac-Toe game using Python.
- To validate player moves and detect winning conditions.
- To display the game result (Win or Draw).

Algorithm

1. Start the program.
2. Create a 3×3 game board.
3. Display the empty board.
4. Allow Player X and Player O to enter their moves alternately.
5. Validate the entered position.
6. Update the board with the player's symbol.
7. Check for a winning condition after each move.
8. If a player wins, display the winner.
9. If all cells are filled without a winner, declare a draw.

10. Stop the program.

Flow Chart



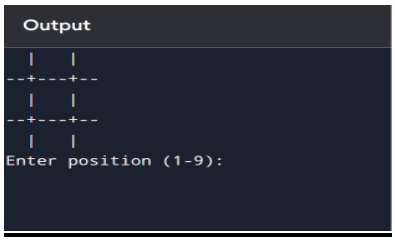
Python Program

```
board = [' '] * 9


def show():
    print(board[0], "|", board[1], "|", board[2])
    print("--+---+--")
    print(board[3], "|", board[4], "|", board[5])
    print("--+---+--")
    print(board[6], "|", board[7], "|", board[8])


def win(p):
    w = [(0,1,2),(3,4,5),(6,7,8),
         (0,3,6),(1,4,7),(2,5,8),
         (0,4,8),(2,4,6)]
```

```
    return any(board[a]==board[b]==board[c]==p for a,b,c in w)

player = 'X'

for i in range(9):
    show()
    pos = int(input("Enter position (1-9): ")) - 1

    if board[pos] == ' ':
        board[pos] = player

        if win(player):
            show()
            print(player, "Wins!")
            break

        player = 'O' if player == 'X' else 'X'
    else:
        print("Position already filled")
else:
    show()
```

```
print("Match Draw")
```

Sample Output



Result

Thus, the Python program to implement the Tic-Tac-Toe Game was executed successfully, and the winner (or draw) was displayed correctly.

Conclusion

The Tic-Tac-Toe game demonstrates basic Artificial Intelligence concepts such as game representation, move validation, and win detection. It helps in understanding turn-based game logic and decision-making in Python

13) MINIMAX ALGORITHM FOR GAMING

Aim

To develop a Python program to implement the Minimax Algorithm for decision-making in a two-player game.

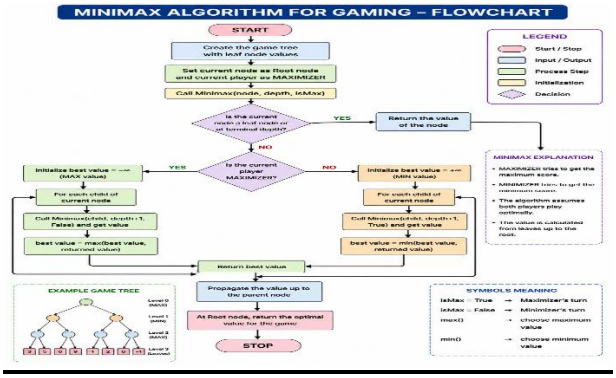
Objectives

- To understand the Minimax Algorithm in Artificial Intelligence.
- To determine the best move for a player in a game.
- To maximize the player's score while minimizing the opponent's score.
- To implement the Minimax algorithm using Python.

Algorithm

1. Start the program.
2. Define the game tree with leaf node values.
3. Set the current player as Maximizer.
4. If the current node is a leaf node, return its value.
5. If the player is Maximizer, choose the maximum value among child nodes.
6. If the player is Minimizer, choose the minimum value among child nodes.
7. Repeat recursively until the root node is evaluated.
8. Display the optimal value.
9. Stop the program.

Flow Chart



Python Program

```
def minimax(depth, node, isMax, values):

    if depth == 3:
        return values[node]

    if isMax:
        return max(minimax(depth+1, node*2, False, values),
                   minimax(depth+1, node*2+1, False, values))
    else:
        return min(minimax(depth+1, node*2, True, values),
                   minimax(depth+1, node*2+1, True, values))

values = [3, 5, 6, 9, 1, 2, 0, -1]

print("Optimal Value:", minimax(0, 0, True, values))
```

Sample Output

```
Output
Optimal Value: 5

=== Code Execution Successful ===
```

Result

Thus, the Python program to implement the Minimax Algorithm was executed successfully, and the optimal value for the game was obtained.

Conclusion

The Minimax Algorithm is a fundamental Artificial Intelligence search algorithm used in two-player games. It helps the maximizing player choose the best possible move while assuming the opponent plays optimally. The algorithm is widely used in games such as Tic-Tac-Toe, Chess, and Checkers for optimal decision-making